

Herald



Herald — Fixed

Field	Value
Revision	20
Created	2026-05-20
Last modified	2026-05-30
Status	active
Status summary	r20: lands HRD-146 (pherald listen dispatch hardening — 120s timeout + process-group kill on the claude_code dispatcher, a975d8b) + HRD-147 (governance dead-letter subscriber + DeadLetterSink + real emitter wiring in boot.go, 9ef4be9) Issues→Fixed — both completed this session with -race tests. Prior r19 (v1.0.0 closure-migration) atomically lands (§11.4.19) 51 code-verified HRDs Issues→Fixed after a re-verification audit (grep/read the impl + run the test + confirm the matching e2e invariant + confirm the docs/qa/<run-id>/ dir; nothing migrated on self-reported status alone). The 51: the 7 flavor constitution bindings HRD-019..025 (<flavor>/internal/bindings Pipelines; 7/7 bindings test suites PASS -race; e2e E90-E96; per-HRD docs/qa/); the entire §43 command catalogue HRD-029..056 (30 commands wired into Cobra roots via registerGitOps/registerSysOps/etc., cmd-test suites PASS, e2e E97-E122, per-HRD docs/qa/); the Batch-D commons_infra.pgxTaskRepository ops HRD-085..089 (all 15 methods real impls not ErrUnsupported, migration 000013_task_resource_snapshots, e2e E123, docs/qa/HRD-085-089-*); Wave 7 HRD-116/118/119/120/121 (qaherald Slack MessengerClient + e2e E124-E131 + paired §1.1 gate tests/test_wave7_mutation_meta.sh + multi-channel §11.4.85 stress/chaos + MESSENGER_CHANNELS.md guide; all hermetic tests PASS); ATMOSphere features HRD-148/149/151/152/153/154 (commons_workable+commons_watch build+test PASS, change→CloudEvent→Runner bridge + inbound action registry + pherald watch daemon + quiet-hours resolver, e2e E139 TestRunWatch_EndToEndOutbound PASS hermetic). NOT migrated (evidence-gated, left in Issues.md): HRD-115 (Slack channel adapter — operator-gated on LIVE docs/qa/HRD-115-LIVE-*/ round-trip evidence per §107.x; only hermetic transcript exists, E127 LIVE-Slack SKIP-with-reason — closing now would be a §107 PASS-bluff); HRD-117 (spec V3 doc-only — §106); HRD-131/150/155/156/157/158 (externally gated); HRD-146/147/081/008/015

Field

Value

(not-started/upstream/operator). Prior r18 atomically lands (§11.4.19) HRD-090 dead-letter handling Completed→Fixed —
commons_infra.pgxTaskRepository.MoveToDeadLetter (was the last Batch-D ErrUnsupported stub): migration
000014_dead_letter_tasks + tx-atomic JSONB-snapshot move + terminal-mark; new 13th OPERATIONAL constitution event class
ClassQueueDeadLetter + QueueEvent + DeadLetter() emit (wired nil-tolerantly; governance §42.2 set unchanged). Four-layer closure: 3 unit + real-PG integration (3 sinks) + §11.4.85 stress/chaos (50 concurrent race-clean, atomic under fault) + paired §1.1 mutation gate (4/0) + e2e E138; commits 549261d+9c765f6; evidence docs/qa/HRD-090-20260529T052228Z/ + docs/qa/HRD-090-stress-chaos-20260529T054500Z/.
Governance-plane subscriber follow-up = HRD-147 (Issues.md).** Prior r17 atomically lands (§11.4.19) the completed Wave 8 Track B (MTPProto user-account harness for §11.4.98 full-automation testing) — HRD-139 (MTPProto Client live wiring + qaherald mtproto CLI), HRD-140 (Subscribe roundtrip), HRD-141 (Wave 6 closed loop), HRD-142 (Wave 6.5 lifecycle) all New→Fixed (these HRDs were tracked directly in the TaskList rather than Issues.md and graduate directly into Fixed.md per §11.4.19 the moment closure landed).** Wave 8 Track B closure proves §11.4.98 (full-automation anti-bluff mandate) end-to-end: 3 MTPProto-driven live tests (E135 Subscribe / E136 Wave 6 closed loop / E137 Wave 6.5 lifecycle) all PASSING LIVE under §11.4.98-pure interpretation, captured under docs/qa/HRD-LIVE-MTPROTO-20260528T125321Z/. Three legacy NON-COMPLIANT tests graduate to §11.4.90 Obsolete in Issues.md r36 with Obsolete-Details cite of §11.4.98 + §108.m superseding-mandate:
TestSubscribe_LiveBotAPI, tests/test_wave6_live_loop.sh, lifecycle --manual mode. v0.6.0 tag (844f340 annotated) was already cut on d843ddf covering Wave 8 Track B closure + the §11.4.98+§11.4.99 anchoring + 9-Helix-submodule §11.4.98+§11.4.99 propagation. Prior r16 atomically migrates (§11.4.19) the completed Tgram-completeness wave —
HRD-133/134/135/136/137/138 (all 6 Tgram-completeness HRDs)
Issues→Fixed.** Wave A: HRD-135 lifecycle handlers (fe2a7c8 — OnEdited / OnMyChatMember / serviceHandler / ErrBotKicked sentinel) + HRD-136 webhook secret_token validator (6d7d543 — crypto/subtle.ConstantTimeCompare, 401-no-leak posture) + HRD-138 operator deep-dive guide (ff9b9c0 — 810-line docs/guides/TELEGRAM.md, 11 §-sections, 3 siblings). Wave B: HRD-133 token-redaction symmetry (8162d14 — sanitizeTgramError wraps 6 error-return sites in Send/SendReply) + HRD-134 429 retry_after honored (8162d14 combined — withRetry429 wraps all 3 telebot bot . Send sites, telebot.FloodError typed extraction, ctx-aware sleep, 3-attempt 60s-clamp). Wave C: HRD-137 stress+chaos suite (this commit — TestTgram_Stress_MultiRecipientFanOut 20×20=400 calls/20 chats deterministic +
TestTgram_Chaos_GetUpdatesPollerResilience SKIP→PASS conversion via one-line subscribe.go baseURL seam fix; 8-phase fault matrix deterministic across -race -count=3 — every iteration IDENTICAL recovery=6 fault(500=12 hang=8 close=3 ok=6) goroutines 3→4). Full tgram suite **26/26 PASS, 0 SKIP** post-Wave-C. All 6 HRDs carry docs/qa/HRD-NNN-20260528T*Z/transcript.txt evidence per §107.x. Prior r15 atomically migrated (§11.4.19) the completed GAP-3 §11.4.85 stress+chaos suite — HRD-122..HRD-130 (Task→Completed) — and the HRD-132 Runner idempotency-window fix (Bug→Fixed): 10 rows
Issues→Fixed.** GAP-3 commits 02d272d→d2a60d3 (HRD-122 commons/

Field	Value
	stresschaos/ scaffold, HRD-123 Gin /v1/events, HRD-124 /v1/compliance+/v1/safety_state, HRD-125 Runner — surfaced HRD-132, HRD-126 pherald listen inbound, HRD-127 claude_code dispatch, HRD-128 container/resource §12.6-safe, HRD-129 e2e E81-E88, HRD-130 paired §1.1 mutation gate); HRD-132 fix 354b883 (claim-before-dispatch at Runner Stage 2 — INSERT events_processed ... ON CONFLICT DO NOTHING is the authoritative exactly-once dispatch gate; latent runner .go:132 CachedRcpt .WasReplay race closed; HRD-125 assertion upgraded to sends==1) + wave3-M4-retire 4f45f26. Full scripts/ e2e_bluff_hunt.sh 78 PASS / 0 FAIL; HRD-130 gate 6 PASS / 0 FAIL (5 load-bearing mutations); HRD-132 evidence docs/qa/HRD-125-stress-chaos-2026-05-27T131251-gap3/ (dispatch_exactly_once=1). HRD-131 (SQLite SSoT migration Phase 1) remains OPEN. Prior r14 captured Wave 6 (pherald inbound runtime + Claude Code headless bridge — code-doc closure). HRD-100 closed atomically with 12 Wave-6 commit SHAs + 8 new e2e invariants E63-E70 (currently SKIP-with-documented-reason — converting to PASS once operator runs the live closed-loop at T10b and commits docs/qa/HRD-100/). Tag v0.4.0 DEFERRED to T13b (operator-supplied live evidence per §107.x — code-doc closure ships in this commit; runtime evidence lands separately).** Wave 6 surfaces: new Cobra subcommand pherald listen (Telegram getUpdates long-poll + OnText/OnPhoto/OnDocument/OnVoice + bot self-filter + sha256 content-addressed attachment download); Opus model pin in spawned claude argv (--model claude-opus-4-7); envelope pre-text — verbatim operator wording "We have received new message from our communication channel" preceding the <<<HERALD-DISPATCH-v1>>> structured block; <<<HERALD-REPLY>>> parser with action routing (reply / issue.open / event.emit); tgram.Adapter.SendReply wiring reply_to_message_id; --qa-out-dir JSONL journaling per §107.x. Wave 6 mutation gate (tests/test_wave6_mutation_meta.sh) lands 3 paired hermetic mutations (M1 blank bot self-filter → echo-loop signature; M2 Opus → Sonnet model swap → E67 FAIL; M3 drop opts .ReplyTo in SendReply → E68 FAIL). Prior r13 captured Wave 4b (TOON application/ toon content negotiation; tag v0.3.0). r12 captured Wave 4a (HTTP/3 + Brotli + Alt-Svc + TLS 1.3; tag v0.2.0). r11 closed HRD-016 (Wave 3b pherald Runner live). r10 closed HRD-011 (Telegram live).
Issues	see Issues.md — r43 (Issues.md) captures this r19 §11.4.19 atomic migration of 51 code-verified HRDs (flavor bindings HRD-019..025, §43 catalogue HRD-029..056, Batch-D pgx HRD-085..089, Wave 7 HRD-116/118/119/120/121, ATMOSphere features HRD-148/149/151/152/153/154); 17 items remain open in Issues.md.
Issues summary	17 still open in Issues.md: HRD-008, HRD-015, HRD-081 (operator/upstream); HRD-115 (Slack adapter — operator-gated on LIVE round-trip evidence per §107.x); HRD-117 (spec doc-only); HRD-131, HRD-150, HRD-155, HRD-156, HRD-157, HRD-158 (ATMOSphere/SSoT externally gated); HRD-146, HRD-147 (dispatch-hardening / governance-subscriber follow-ups); HRD-143/144/145 (Obsolete).
Fixed	HRD-001..HRD-007, HRD-009, HRD-009b, HRD-010, HRD-011, HRD-012, HRD-013, HRD-014, HRD-016, HRD-017, HRD-018, HRD-019..HRD-025, HRD-026, HRD-027, HRD-028, HRD-029..HRD-056, HRD-080, HRD-085..HRD-089, HRD-090, HRD-092, HRD-093, HRD-094, HRD-095, HRD-096, HRD-097, HRD-098, HRD-099, HRD-100, HRD-110, HRD-111, HRD-112, HRD-113, HRD-114, HRD-116, HRD-118, HRD-119, HRD-120,

Field	Value
	HRD-121, HRD-122, HRD-123, HRD-124, HRD-125, HRD-126, HRD-127, HRD-128, HRD-129, HRD-130, HRD-132, HRD-133, HRD-134, HRD-135, HRD-136, HRD-137, HRD-138, HRD-139, HRD-140, HRD-141, HRD-142, HRD-146, HRD-147, HRD-148, HRD-149, HRD-151, HRD-152, HRD-153, HRD-154
Fixed summary	v1.0.0 closure-migration (r19 — 51 HRDs: flavor constitution bindings HRD-019..025, §43 command catalogue HRD-029..056, Batch-D pgxTaskRepository ops HRD-085..089, Wave 7 multi-channel HRD-116/118/119/120/121, ATMOSphere workable-items features HRD-148/149/151/152/153/154 — each re-verified against committed code + passing tests + e2e invariant + docs/qa before migration). Wave 8 Track B (HRD-139/140/141/142 — MTPProto user-account harness proves §11.4.98 full-automation end-to-end with 3 live MTPProto-driven tests PASSING LIVE 2026-05-28; v0.6.0 tag covers the closure); Tgram-completeness wave (HRD-133/134/135/136/137/138 — 6 HRDs across 3 sub-waves A/B/C, full tgram suite 26/26 PASS 0 SKIP); GAP-3 §11.4.85 stress+chaos suite (HRD-122..HRD-130 Completed) + HRD-132 Runner idempotency-window fix (claim-before-dispatch, exactly-once dispatch <code>sends==1</code>); spec V1→V3 r9; Go module foundation + Foundation M1/M2/M3; HRD-010 commons_storage live wiring; HRD-011 Telegram live (live message_id=5 evidence 2026-05-22); HRD-012 Claude Code dispatcher live; HRD-016 pherald /v1/events Runner wiring (Wave 3b live — 7-stage pipeline + e2e E37-E42); universal §11.4.73 + §11.4.74 mandates propagated; I6 gate refined; Wave 2 flavor scaffolds (HRD-092..097) closed atomically; Wave 3a substrate (HRD-099 commons_auth JWT verifier) + 2 live REST routes (HRD-028, HRD-098) closed atomically.

Continuation see CONTINUATION.md.

Table of contents

- [Recently fixed](#)

Recently fixed

ID	Type	Criticality	Title
HRD-147	task	low	Governance-plane dead-letter subscriber — durable consumer draining .queue events into a DeadLetterSink (MemoryDeadLetterSink M1; a PostgresDeadLetterSink is a drop-in behind the interface). commons_boot.go now wires the REAL emitter into newPgxTaskRepositoryWith the nil-emitter) + stands up a DeadLetterSubscriber drain goroutine (Close error-counted), guarded to build only alongside the Postgres pool and to fall back if emitter construction fails (a dead-letter move must never fail because the audit Mirrors the commons_constitution.AuditStore append-only pattern tests (move→emit→subscribe→sink end-to-end, failure-count propagation, 500ms - race, idempotent Close) + the 3 existing MoveToDeadLetter tests PASS. M2 PostgresDeadLetterSink behind the interface is the documented follow-up pherald_listen dispatch hardening — bounded per-message resource use inbound→claudcode.Dispatch path. The production path threaded an ctx (pherald_listen gctx → subscriber goroutine → cmd.Output()), so child could block the subscriber indefinitely (HRD-146 confirmed reachable, not Added DefaultDispatchTimeout=120s + HERALD_CLAUDE_DISPATCH
HRD-146	task	middle	

ID	Type	Criticality	Title
HRD-019	task	high	env knob + SetDispatchTimeout(); Dispatch wraps the caller ctx in m deadline, budget) (a tighter caller still wins; cancellation is not swallow setProcessGroup (Setpgid + Cmd.Cancel→kill(-pgid) + WaitDelta holding child can't wedge Output() past the deadline. §11.4.85 evidence: a re 30 shim is KILLED in 702ms (vs 30s) with the timeout-named error; caller-can 402ms; happy-path 26ms. - race - count=3 green. cherald constitution bindings — cherald/internal/bindings Pipeline cherald-owned §42.3 rules as concrete commons_constitution.Evaluator through 5 event classes (.policy.violation/.gate.failed/.gate.recovered/.credential over the Batch-A emit+persist+audit foundation; POST /v1/compliance/ surface drives detect→emit→persist→audit, visible on the existing GET /v1/ pull surface. Vertical-slice bespoke detectors landed (§11.4.29 naming, §11.4.10 §11.4.44 doc-revision-header, §11.4.18 script-docs, §11.4.73 spec-drift, §11.4.7 remaining 25 rules use the shared marker-check mechanism (verdict decided up catalogue command / sweep) with the full emit-routing framework in place — c remaining bespoke detector is per-rule follow-up.
HRD-020	task	high	sherald host-safety + repo-safety constitution bindings — sherald/inte bindings Pipeline registers the 15 sherald-owned §42.3 host/repo-safety rule §11.4.24/.26/.32/.36/.41/.47/.71, §12.1/.2/.3/.6) as concrete commons_constitution.Evaluator impls routed through 6 SAFETY e (.host.safety.breach/.repo.safety.breach/.gate.recovered over the Batch-A emit+persist+audit foundation, mirroring the proven HRD-019 pattern. The 3 named DETECTION hooks landed as REAL pure classifiers — c detector (§9.1: rm/git-reset/clean without backup → .repo.safety.breach interceptor (§11.4.41/§9.2: force-push without merge-first OR session-auth → .repo.safety.breach), mem-budget watcher (§12.6: reported used_fract .host.safety.breach) — plus forbidden-host-op (§12.1) + prereq-marke remaining rows. CRITICAL §12-safety: every detector DETECTS+REPORTS string description; NONE performs the destructive op, force-pushes, suspends th allocates memory (the mem-budget watcher reads a fabricated fraction — no re §43 command bodies (HRD-033/046/056) supply the live op-interception feedin
HRD-021	task	middle	bherald CI/test bindings — bherald/internal/bindings Pipeline re bherald-owned §42.3 CI/test gate-result rules (§1, §1.1, §11.4.2/.3/.4/.5/.7/.9/.13/.14/.24/.27/.30/.39/.43/.46/.48–.52/.67) as concrete commons_constitution.Evaluator impls routed through 3 event class (.gate.failed/.gate.recovered via the build/test gate-result class + .policy.violation + .repo.safety.breach for the shared §11.4.30 row) over the Batch-A emit+persist+audit foundation, mirroring the proven HR Pipeline pattern. bherald is the BUILD/CI flavor; it emits CI/test gate-result con that integrate with the consuming project's CI workflow. 3 bespoke pure detecto result classification (§1/§11.4.50 — PASS/FAIL/flaky from a recorded gate outo .gate.failed/.gate.recovered), test-tier-verify (§11.4.27/§40.2 — th matrix; a missing tier FAILs), anti-bluff-PASS detection (§11.4.2/.5 — a gate th WITHOUT a captured-evidence artefact is a §11.4 PASS-bluff → FAIL). Rema shared marker-check mechanism (verdict decided upstream by the §43 comman full emit-routing framework in place.
HRD-022	task	middle	rherald release constitution bindings — rherald/internal/binding registers the 4 rherald-owned §42.3 release-lifecycle rules (§4 tag mirroring, §5 multi-format export [shared with cherald], §11.4.38 installable-asset evidence, § retest before release tag) as concrete commons_constitution.Evaluator

ID	Type	Criticality	Title
			through 2 event classes (<code>.release.gate.blocked</code> via the existing <code>ReleaseGateBlocked</code> emitter surface for §4/§11.4.38/§11.4.40 + <code>.policy.violation</code> for the shared §5 changelog row; a release recovery maps to the shared <code>.gate.blocked</code> companion since the release class family has no distinct "recovered" leaf) over the <code>emit+persist+audit</code> foundation, mirroring the proven HRD-019/020/021 Pipeline through the RELEASE flavor. 4 bespoke PURE detectors landed: tag-mirror-parity (§43 — parent but missing on any owned mirror, <code>with_tag<mirrors></code> , → <code>.release.gate.blocked</code>), changelog-conformance (§5 — non-Conventional changelog OR stale multi-format export → <code>.policy.violation</code>), pre-tag-race (§11.4.40 CRITICAL — tag attempted without a green all-8-§40.2-tier retest → <code>.release.gate.blocked</code>), installable-asset-evidence (§11.4.38 — release install check → <code>.release.gate.blocked</code>). CRITICAL §12-safety: every CLASSIFIES a recorded release-op outcome string; NONE tags, pushes, force-downloads, or installs — the live release-op interception is scope-locked to the (HRD-031/032/045).
HRD-023	task	middle	pherald project bindings (§2 commit+push, §3 submodule propagation, §11.4.11/.15/.21/.22/.34/.37/.42/.55/.66/.71/.74) — Batch B: pherald/internal/bindings/ Pipeline + §42.3-owned rule set + 5 bespoke PURE detectors (consequence discipline / submodule-propagation-order / pre-push-fetch-guard / fetch-before-push) through the commons_constitution Evaluator+audit foundation; -race -count=3 Live op-interception deferred to §43 HRD-029/030/043/044/049/053.
HRD-024	task	middle	iherald constitution-rule escalation bindings (§11.4.10/.10.A credential-leak + §11.4.66 operator-blocked escalation) — Batch B: iherald/internal/bindings/ Pipeline routing .credential.leak (forced-critical) + .policy.violation detectors wired through commons_constitution Evaluator+audit; -race -count=3 no-real-secret attestation. Live paging deferred to §43 / HRD-024-paging.
HRD-025	task	low	scherald scheduled-audit bindings (§11.4.45 periodic Status.md sweep + daily compliance digest) — Batch B: scherald/internal/bindings/ Pipeline .policy.violation + 3 bespoke detectors (status-sweep / digest-cadence) through commons_constitution Evaluator+audit; -race -count=3 green, e2e E96 scheduler deferred to §43 HRD-047.
HRD-029	task	middle	§2 pherald commit-push — single-entripoint locked commit + multi-mirror Batch C: implemented as gitops command (§11.4.74 wrap of git binary); --emit feeds hermetic temp-repo tests -race green
HRD-030	task	middle	§3 pherald submodule propagate — owned-submodule walk in propagate Batch C: implemented as gitops command (§11.4.74 wrap of git binary); --emit feeds binding; hermetic temp-repo tests -race green
HRD-031	task	middle	§4 rherald tag mirror — assert tag exists on every owned submodule — implemented as rherald §43 release command (rherald/cmd/rherald/release_cmds.go) feeding the HRD-022 rherald binding; reuses shared commons/hermetic -race green
HRD-032	task	low	§5 rherald changelog generate — Conventional Commits → docs/ <v>.md + multi-format export — Batch C: implemented as rherald §43 release command (rherald/cmd/rherald/release_cmds.go) feeding the HRD-022 rherald binding; reuses shared commons/gitops; hermetic -race green
HRD-033	task	high	§9.1 sherald destructive guard <op> — wrap rm/git-reset/git-push prerequisite checks — Batch C: implemented as sherald §43 command (sherald/sysops_cmds.go) feeding the HRD-020 sherald binding; PRE-EXECUTE (detects+classifies+exits-nonzero, never performs the op); hermetic -race green; fix so the CLI runs without a JWT secret
HRD-034	task	middle	

ID	Type	Criticality	Title
			§9.3 sherald backup-snapshot <target> — hardlinked snapshot helper; straggler: implemented in sherald/cmd/sherald/sysops_cmds.go (detect-only; refuses overwrite, classifies §9.3 SubjectBackup → .gate.recovered); hermetic equality -race green
HRD-035	task	middle	§11.4.2 bherald evidence capture <test_id> — captured-evidence Batch C: implemented as bherald §43 build/CI command (bherald/cmd/bherald/build_cmds.go) feeding the HRD-021 bherald binding; hermetic -race green
HRD-036	task	high	§11.4.10 cherald creds scan — gitleaks/trufflehog integration, emits .credential.leak — Batch C: implemented as cherald §43 verify/check command (cherald/cmd/cherald/checkops_cmds.go) feeding the HRD-019 cherald binding; detect-only; hermetic -race green (routed to §11.4.10 credential-leak; findings Report secret value printed)
HRD-037	task	low	§11.4.12 cherald docs sync — regen Issues_Summary / Fixed_Summary / Status_Summary — Batch C: implemented as cherald §43 docs-pipeline command (cherald/cmd/cherald/docops_cmds.go) feeding the HRD-019 cherald binding; --apply to regenerate; ships the §107 lazy-verifier fix so the CLI runs without hermetic -race green
HRD-038	task	low	§11.4.18 cherald script-docs check — assert sibling .md for every script Batch C: implemented as cherald §43 verify/check command (cherald/cmd/cherald/checkops_cmds.go) feeding the HRD-019 cherald binding; detect-only; hermetic (routed to §11.4.60 — §11.4.62 has no cherald rule)
HRD-039	task	low	§11.4.19 / .23 cherald fixed align+cherald colorize — Issues_Summary HTML colorizer — Batch C: implemented as cherald §43 docs-pipeline command (cherald/cmd/cherald/docops_cmds.go) feeding the HRD-019 cherald binding; --apply to regenerate; ships the §107 lazy-verifier fix so the CLI runs without hermetic -race green
HRD-040	task	high	§11.4.26 sherald constitution pull — wrap fetch + rebase + validate .bundle.updated — Batch C: implemented as sherald §43 command (sherald/cmd/sherald/sysops_cmds.go) feeding the HRD-020 sherald binding; PRE-FLIGHT GA (detects+classifies+exits-nonzero, never performs the op); hermetic -race green; fix so the CLI runs without a JWT secret
HRD-041	task	middle	§11.4.27 bherald test-tier verify — 8-tier matrix verification — Batch C: implemented as bherald §43 build/CI command (bherald/cmd/bherald/build_cmds.go) feeding the HRD-021 bherald binding; hermetic -race green
HRD-042	task	low	§11.4.31 cherald submanifest verify — Submodule-Dependency-Matrix Batch C: implemented as cherald §43 verify/check command (cherald/cmd/cherald/checkops_cmds.go) feeding the HRD-019 cherald binding; detect-only; hermetic
HRD-043	task	high	§11.4.36 pherald install-upstreams — install_upstreams wrapper — Batch C: implemented as gitops command (§11.4.74 FindScript-resolved constitution script) feeding the HRD-023 binding; hermetic temp-repo tests -race green
HRD-044	task	middle	§11.4.37 pherald fetch-guard — pre-edit fetch + rebase enforcement — Batch C: implemented as gitops command (§11.4.74 wrap of git binary); --emit feeds HRD-023 binding; hermetic temp-repo tests -race green
HRD-045	task	high	§11.4.40 rherald gate retest — pre-tag full-suite retest gate — Batch C: implemented as rherald §43 release command (rherald/cmd/rherald/release_cmds.go) feeding the HRD-022 rherald binding; reuses shared commons/gitops; hermetic -race green
HRD-046	task	high	§11.4.41 sherald force-push gate — merge-first + per-session-auth enforcement Batch C: implemented as sherald §43 command (sherald/cmd/sherald/sysops_cmds.go) feeding the HRD-020 sherald binding; PRE-FLIGHT GA

ID	Type	Criticality	Title
			(detects+classifies+exits-nonzero, never performs the op); hermetic -race green; fix so the CLI runs without a JWT secret
HRD-047	task	middle	§11.4.45 / .56 <code>scherald status-digest</code> — periodic Status.md sweep + regen — Batch C straggler: implemented in <code>scherald/cmd/scherald/digest.go</code> feeding the HRD-025 <code>scherald</code> binding (§11.4.45 status-sweep; detect/report + regenerates Status_Summary); hermetic -race green
HRD-048	task	low	§11.4.53 <code>cherald fixed-summary sync</code> — standalone Fixed_Summary Batch C: implemented as <code>cherald</code> §43 docs-pipeline command (<code>cherald/cmd/cherald/docops_cmds.go</code>) feeding the HRD-019 <code>cherald</code> binding; detect-by-default + regenerate; ships the §107 lazy-verifier fix so the CLI runs without a JWT secret green
HRD-049	task	middle	§11.4.55 <code>pherald reopen <HRD-NNN></code> — Issues→Fixed reversal + Reopen Batch C: implemented as <code>gitops</code> command (the Wave 6.5 inbound Fixed→Issues emit feeds HRD-023 binding; hermetic temp-repo tests -race green
HRD-050	task	middle	§11.4.59 <code>cherald readme sync</code> — README doc-link regen + multi-form Batch C: implemented as <code>cherald</code> §43 docs-pipeline command (<code>cherald/cmd/docops_cmds.go</code>) feeding the HRD-019 <code>cherald</code> binding; detect-by-default + regenerate; ships the §107 lazy-verifier fix so the CLI runs without a JWT secret green
HRD-051	task	high	§11.4.60 <code>cherald composite-gate</code> — canonical implementation of CM-COMPOSITE-SYNC — Batch C: implemented as <code>cherald</code> §43 verify/check command (<code>cherald/cmd/cherald/checkops_cmds.go</code>) feeding the HRD-019 <code>cherald</code> binding; detect-only; hermetic -race green
HRD-052	task	middle	§11.4.65 <code>cherald export</code> — bulk Markdown export wrapper (md/html/pdf) implemented as <code>cherald</code> §43 docs-pipeline command (<code>cherald/cmd/cherald/docops_cmds.go</code>) feeding the HRD-019 <code>cherald</code> binding; detect-by-default + regenerate; ships the §107 lazy-verifier fix so the CLI runs without a JWT secret green
HRD-053	task	high	§11.4.71 <code>pherald pre-push</code> — fetch + investigate + integrate hook — Batch C as <code>gitops</code> command (§11.4.74 wrap of git binary); --emit feeds HRD-023 binding; repo tests -race green
HRD-054	task	middle	§11.4.73 <code>cherald spec-version check</code> — Revision-vs-edits drift detection implemented as <code>cherald</code> §43 verify/check command (<code>cherald/cmd/cherald/checkops_cmds.go</code>) feeding the HRD-019 <code>cherald</code> binding; detect-only; hermetic -race green
HRD-055	task	middle	§11.4.74 <code>cherald catalogue-check <pr></code> — scan PR for Catalogue-Checker runner over vasic-digital + HelixDevelopment — Batch C: implemented as <code>cherald</code> §43 check command (<code>cherald/cmd/cherald/checkops_cmds.go</code>) feeding the HRD-019 <code>cherald</code> binding; detect-only; hermetic -race green
HRD-056	task	high	§12.6 <code>sherald mem-budget watch</code> — daemon-mode 60% threshold watch for <code>.host.safety_breach</code> — Batch C: implemented as <code>sherald</code> §43 command (<code>cmd/sherald/sysops_cmds.go</code>) feeding the HRD-020 <code>sherald</code> binding; GATE (detects+classifies+exits-nonzero, never performs the op); hermetic -race green; ships the §107 lazy-verifier fix so the CLI runs without a JWT secret
HRD-085	task	middle	Implement single-task admin operations on <code>commons_infra.pgxTaskRepository</code> GetByID, Update, Delete. Required for Requeue + MoveToDeadLetter cache operator tooling. Currently all three return <code>ErrUnsupported</code> (HRD-085). BackgroundTask row by ID; full-row Update including JSONB columns; soft-delete decision per §107.
HRD-086	task	middle	Implement worker-side running-task state mutations on <code>commons_infra.pgxTaskRepository</code> — UpdateStatus, UpdatePriority

ID	Type	Criticality	Title
			UpdateHeartbeat, SaveCheckpoint. Required by River workers + the reaper. Currently all four return ErrUnsupported (HRD-086). Scope: status guarded by valid-state-machine check; progress as (float64, message); last updated_at = now() writeback; checkpoint as opaque []byte blob persisted in task row.
HRD-087	task	low	Implement admin / stats / audit reads on commons_infra.pgxTaskRepository. GetByStatus, GetPendingTasks, CountByStatus, GetTaskHistory. v1/queue/* introspection endpoints + Peek delegate. Currently all four return ErrUnsupported (HRD-087). Scope: status-filtered paginated reads; pending Peek; status-bucketed counts; per-task execution-history rows from the task_execution_history table.
HRD-088	task	middle	Implement queue-recovery reads on commons_infra.pgxTaskRepository. GetStaleTasks, GetByWorkerID. Drives the stuck-task reaper + worker-loop. Currently both return ErrUnsupported (HRD-088). Scope: stale-tasks against updated_at; per-worker assignment lookup for re-dispatch after a worker dies.
HRD-089	task	low	Implement resource-monitor I/O on commons_infra.pgxTaskRepository. SaveResourceSnapshot, GetResourceSnapshots. Persists per-task resource samples for capacity-planning + post-mortem. Currently both return ErrUnsupported (HRD-089). Scope: append-only insert keyed by (task_id, captured_at). paginated read in reverse-chronological order. Schema target: a task_resource table not yet in the §9.6 migration set — adding it is part of this HRD.
HRD-116	task	middle	Wave 7 T7 — qaherald Slack MessengerClient + channel-keyed builder (commons/internal/messenger/{slack.go,builder.go}): raw-HTTP client implementing 11 MessengerClient methods (Me/Send/SendPhoto/SendDocument/SendVideo/WaitForReply/Download/Preflight/GetChatMember/Close); GetUpdates via conversations.history+oldest=<ts> watermark (Slack has no getConversationMessageID = dotted-float ts string; channel-keyed Build(BuildConfig) unknown→error). Token NEVER in errors — sanitizeError() defense + TestSlack_Send_ErrorDoesNotLeakToken pins it with a planted xoxb-SUPERSECRET-must-not-leak-7777. 30 hermetic httpstest tests -race time pin var _ MessengerClient = (*SlackClient)(nil). KNOCKOUT CLEANUP: ships files.upload V1 (deprecated by Slack 2025-Q1 in favor of files.upload_v2) — functionally equivalent for the httpstest seam, swappable without API breakage in slack.go header.
HRD-118	task	high	Wave 7 T9 — e2e_bluff_hunt.sh E124-E131 multi-channel invariants: E124 channel resolution, E125 tgram self-filter post-Wave-7 (pure-refactor invariant), E126 slack (interface satisfied + auth.test + chat.postMessage wire bytes + registry wiring), Slack (env-gated HERALD_SLACK_BOT_TOKEN+HERALD_SLACK_CHANNEL_ID+TestSlack triple-guard to prevent silent "no tests to run" PASS-bluff), E128 per-channel invariants, E129 generalized IsSelfEcho (Username+UserID+empty-self), E130 multi-channel wiring, E131 qaherald messenger.Build dispatch. sc_anchor on the T6+T7 common.
HRD-119	task	high	Wave 7 T10 — paired §1.1 mutation gate tests/test_wave7_mutation_gate (PASS / 0 FAIL): M1 blank IsSelfEcho → TestIsSelfEchoUsername (hazard caught) → restored → PASS; M2 drop slack channels.Register in TestSlackRegisteredViaInit FAILs → restored → PASS; M3 drop slack append in send.go → TestSlackSendReplyUsesThreadTS FAILs → restored + quiescence assertion (no MUTATED W7 - residue in 3 production target files). Reuses Wave 6 canonical .git/MUTATION_IN_PROGRESS lockfile + trap-catch-flight dirty-tree-and-residue guard + byte-for-byte assert_restored. FORK-ONLY (§107.y / no-concurrent-mutation operator mandate).

ID	Type	Criticality	Title
HRD-120	task	high	Wave 7 T11 — §11.4.85 multi-channel stress + chaos. Extended HRD-126's pherald/listen_stress_chaos_test.go (NOT duplicated to a new TestListen_MultiChannel_StressBurst (2 channels × 200 concurrent runListen — pins no-loss/no-double + per-channel attribution via perChannelCountingReplier keyed on rcpt.Channel proving the c NOT cross-wire; throughput ~9-19k ev/s under -race; clean shutdown <3s + no TestListen_MultiChannel_ChaosOneChannelDies_OtherKeep fires 1 event then errors; slack ticks 50 events at 2ms — pins concurrency-of-f via conjunction: ≥5 slack dispatches landed BEFORE fail-loud teardown AND surfaces tagged channel="tgram-stub" + subscribe). 12/12 PASS over -count=3 (3 iterations × 2 new + 2 sibling HRD-126); race-detector clean. run found: fail-loud-tears-down-siblings (per-channel supervised restart is a deferred doc-comment warns against silent change). Evidence anchors: multichannel_no_drop_under_burst=1 (stress) + multichannel_concurrency_proven=1 + multichannel_one_channel_death_surfaced=1 (chaos). Wave 7 T12 — docs/guides/MESSENGER_CHANNELS.md operator guide sections + 3 siblings .html/.pdf/.docx): §1 architecture (commons.Channel outbound commons_messaging/channels.Channel inbound + registry + IsSelfEcho + per-ASCII fan-in diagram), §2 HERALD_CHANNELS selector (fail-loud on unknown Telegram live setup (BotFather → token → chat-id), §4 Slack live setup (api.sl Socket Mode + scopes + xoxb-/xapp- tokens; §107 token-redaction sanitization TestSlack_Send_ErrorDoesNotLeakToken), §5 inbox isolation under {HERALD_INBOX_DIR}/<channel>/<sha256>.<ext>, §6 anti-echo-l (cross-the-wire-then-cache contract), §7 URL schemes for both adapters, §8 14-channel" implementer checklist + companion "what you do NOT touch" list (fr come for free), §9 10-entry troubleshooting cookbook (echo-loop / channel_not invalid_auth / wrong inbox / HERALD_CHANNELS=unknown), §10 6-checkl deploy audit mirroring OPERATOR_CREDENTIALS.md style, §11 references V3 §11/§32.2/§43 + Wave 7 plan + adapter source paths + every HRD-110..121 REGEN_ALL=1 bash scripts/export_docs.sh (new-guide bootstrap
HRD-121	task	low	ATMOSphere integration WS-1 — commons_workable SQLite SSoT foundation (LANDED + tested 2026-05-29). New L1 module: SQLite store (pure-Go mod sqlite, no CGO) mirroring the workable-items schema v2 (items PK (atm_id,current_location) + item_history + meta, WAL, 10-v set); full CRUD repo (Create/GetByID/Update/Delete/List, loud not-found, close validation); per-property Diff(prev,curr) change-feed emitting item.c item.deleted/item.status.changed/item.field.changed/ item.content.updated with exact old→new; ParseTracker matching REAL ## §... [ATM-NNN] + **Status:**/**Type:**/**Severity: real-SQLite tests -race -count=3 green. Keystone for R1/R3/R6.
HRD-148	feature	high	ATMOSphere integration WS-2 — commons_watch file/DB watcher (LANDED 2026-05-29). New L1 module: fsnotify watcher (direct dep) over the .db SSoT debounce-coalesce + SQLite-WAL polling fallback (WAL writes hit the -wal s raise reliable fsnotify Write on the main .db); graceful ctx-cancel, leak-free. 4 -race green. Feeds R1/R2.
HRD-151	feature	high	ATMOSphere integration WS-3 — change→CloudEvent→Runner bridge + Jira per-property-diff message formatter (reuse the existing pherald/internal fan-out; add only the workable-event producer + formatter).
HRD-152	feature	high	

ID	Type	Criticality	Title
			ATMOSphere integration WS-4 — inbound action registry + <code>item.update/investigation.start + ItemMutator</code> (writes <code>commons_workable</code> orchestrator (act-with-confirmation per operator 2026-05-29).
HRD-153	feature	middle	ATMOSphere integration WS-2 — watcher daemon entrypoint (<code>pherald watch→diff→notify</code> loop; promote to a <code>wherald</code> flavor if it grows).
HRD-154	task	middle	ATMOSphere integration WS-3 — enforce preference/quiet-hours routing (<code>QuietHours</code> types exist but the resolver ignores them).
			Dead-letter handling on <code>commons_infra.pgxTaskRepository</code> — Move (was the last <code>ErrUnsupported Batch-D</code> stub). Migration <code>000014_dead_letter</code> (.up/.down, FK→background_tasks ON DELETE CASCADE, no tenant/RLS — Tx-atomic move: INSERT a full JSONB task snapshot into <code>dead_letter_tasks</code> mark <code>background_tasks.status='dead_letter'</code> (soft-mark, NOT preserve the events/snapshots FK children); JSONB passed as <code>string()</code> per t SQLSTATE-22P02 lesson. New 13th OPERATIONAL constitution event class <code>ClassQueueDeadLetter("queue.dead_letter") + QueueEvent</code> payload + <code>DeadLetter()</code> emit method (governance set §42.2's 12 classes unchanged; c 12→13); wired nil-tolerantly into the repo via <code>newPgxTaskRepositoryWi</code> (boot.go keeps the nil-emitter — the Foundation plane has no governance bus+ emitting there would be an emit-into-the-void §107 bluff; governance-plane sub HRD-147 follow-up). Best-effort emit AFTER the durable commit — surfaced swallowed. Four-layer closure (CONST-048) : 3 hermetic unit tests + real-PG (TestRepoMoveToDeadLetter_RoundTripAndEmit, 3 independent si terminal-state + dlq snapshot + real-bus .queue.dead_letter delivery) + chaos (50 concurrent moves race-clean, emit-failure-atomic, cancelled-ctx-no-o §1.1 mutation gate (<code>tests/test_hrd090_mutation_meta.sh</code> , 4/0) + e -race green.
HRD-090	task	middle	Tgram-completeness Wave B — token-redaction symmetry (<code>commons_messaging/channels/tgram/errors.go + send.go</code> wraps). <code>sanitizeTgramError(token) string</code> strips the bot token from every <code>Send/SendReply</code> error message can NEVER appear in observable output — 6 error-return sites wrapped (ensure withRetry429 paths in both <code>Send + SendReply</code>).
HRD-133	task	high	<code>TestTgram_Send_ErrorDoesNotLeakToken</code> plants 123456: super not-leak-9999, drives an <code>httptest</code> 401 path, asserts the token substring is ab returned <code>err.Error()</code> (mirrors slack's <code>TestSlack_Send_ErrorDoesN</code> Unit-level <code>TestTgram_SanitizeTgramError_Unit</code> (4 subtests) pins the captured in the source: %w error-wrap replaced with %s + <code>sanitizeTgramError(err.Error(), token)</code> since <code>strings.Rep</code> operates on the string form — the error chain is sacrificed to guarantee the secret Composes with HRD-134 (the 429 retry's log line + final error are themselves s logging never leaks the token).
HRD-134	task	high	Tgram-completeness Wave B — Bot API rate-limit 429 + <code>retry_after</code> hono (<code>commons_messaging/channels/tgram/send.go + tgram.go</code> sle errors.go::extractRetryAfter).withRetry429(ctx, fn) er telebot bot . Send call sites (<code>Send</code> text body + <code>SendReply</code> text + <code>SendReply</code> pe out). On 429 + <code>retry_after=N</code> → sleep N seconds (<code>ctx.Done()</code> -aware attempts total, clamp per-retry wait to 60s (defense-in-depth against Bot API sp <code>retry_after</code> extracted via <code>errors.As(err, &telebot.FloodError{})</code> .RetryAfter (telebot's typed error — NOT Logged retry line goes through <code>sanitizeTgramError</code> so the token stays ou (composes with HRD-133). Injectable sleeper field on <code>Adapter</code> for hermet + fake-clock both supported via <code>SetSleeperForTest</code>). 4 hermetic <code>httptest</code> t

ID	Type	Criticality	Title
HRD-135	task	middle	(HonorsRetryAfter / GivesUpAfter3Attempts / RespectsCtxCancel / ClampsRet unit test (ExtractRetryAfter_Unit) — 5 total, all -race -count=1 green. Tgram-completeness Wave A — edited-message + bot-kicked typed-error + serv ignore-and-log handlers (commons_messaging/channels/tgram/sub bot.Handle(telebot.OnEdited, ...) builds the same InboundEven does + Raw["edited"]=true; bot.Handle(telebot.OnMyChatMem botKickedFromUpdate detects status transitions to kicked/left/restricted fo specifically (Alice-left + promotion-to-Admin false-positive guarded) and emits log.Printf "tgram: bot kicked from chat: chat_id=... ne ErrBotKicked"; serviceHandler registered against 15 telebot service C (UsersJoined / UserLeft / NewGroupTitle / etc.) emits NOTHING to InboundHa tgram: ignored service message kind=... chat_id=.... Sentinel ErrBotKicked = errors.New("tgram: bot kicked from cha (callable via errors.Is). The OnMyChatMember closure returns nil delibera poller keeps running for OTHER chats. 17/17 PASS -race -count=1 (3 ne subscribe_lifecycle_test.go + 14 existing, zero regression). Tgram-completeness Wave A — Bot API webhook secret_token validation (commons_messaging/channels/tgram/webhook.go). WebhookH http.Handler validates X-Telegram-Bot-API-Secret-Token via cryp subtle.ConstantTimeCompare (timing-attack-safe); refuses empty secr time(NewWebhookHandler("", ...) errors loud — §107 footgun guard); re empty body on header mismatch / missing header (no token-existence-oracle lea 400 on malformed JSON body; 405 on wrong method (POST-only); 500 (no bo InboundHandler error. Forwards a decoded tele.Update as the SAME Inbo shape OnText/OnEdited produce — unified downstream contract. Skeleton pherald listen --webhook + the setWebhook(secret_token=... the public-endpoint TLS listener are deferred to V1.x; the cryptographic-validat tested so the future flip is a one-flag affair. 7/7 hermetic httpstest tests -race - ValidSecret_Dispatches, InvalidSecret_Returns401_NoTokenLeak, MissingSec GET_Returns405, MalformedJSON_Returns400, EmptySecretAtConstruction ConstantTimeCompare. Tgram-completeness Wave C — §11.4.85 stress + chaos suite (commons_mes channels/tgram/stress_chaos_test.go). TestTgram_Stress_MultiRecipientFanOut drives the REAL Adapter.SendReply via NewAdapterWithBaseURL httpstest seam unde iterations = 400 total calls against 20 UNIQUE chat_ids on the SAME *tgram asserts no-loss (every chat_id received exactly 20 hits), no-double (global hit co confusion (uniform per-chat distribution, no cross-wire), -race-clean, throughpu (observed 5215-6422 ev/s, p99=9.73-15.20ms). TestTgram_Chaos_GetUpdatesPollerResilience converted from with-reason to a real httpstest-driven fault-injector PASS via a ONE-LINE subs fix: threads a.baseURL into telebot.Settings.URL before NewBot (p ensureBot pattern at tgram.go: 60-67; production path unchanged when ""). 8-phase fault matrix (3× 500 + 2× 1.5s hangs honoring ctx-cancel + 1× mic connection-close + 2× success), bot-username-aware self-filter bypass (success from @someuser, not chaostestbot). Deterministic across -race -c §11.4.50 — all 3 iterations IDENTICAL: recovery_event_count=6 fault_modes(500=12 hang=8 close=3 ok=6) subscribe_err deadline exceeded goroutines 3→4 (leaked=1) duration= tgram suite 26/26 PASS, 0 SKIP post-fix (was 24/24 with chaos SKIPped). §11 captured-evidence per-iteration table; P2 blast-radius — subscribe.go base only when unused (production unaffected); P3 cross-feature — the seam mirror
HRD-136	task	low	
HRD-137	task	high	

ID	Type	Criticality	Title
HRD-138	task	low	ensureBot pattern, no novel state; P4 deep-research — telebot.v3 LongPoller under fault (poller logs + continues, no internal retry); P5 anti-bluff — handlerF atomics, ctx-aware subscribe return, no goroutine leak (delta=1) all positive cap from REAL Subscribe poller, not synthetic. Tgram-completeness Wave A — docs/guides/TELEGRAM.md per-module dive guide (810 lines, 11 §-sections, 58 subsections, 3 siblings .html/.pdf/.docx) overview (inbound getUpdates + outbound Send/SendReplyGeneric); walkthrough (/newbot → token → /setprivacy); §3 required env table + HERALD_TGRAM_CHAT_ID (3 methods: getUpdates curl / @userinfobot / @F pre-deploy smoke-test commands; §5 attachment size limits (4096 text, 10/50/2 50MB document Bot-API-cap workaround = local Bot API server) + the content path; §6 chat-type matrix (private/group/supergroup/channel id formats + requirements per type); §7 reply threading via reply_to_message_id + the attachment type (Wave 6.5 mutation gate M3); §8 self-filter contract (cross-the-wire getMe + call loop hazard if skipped — Wave 6 T12 M1); §9 12-entry troubleshooting cookbook (401/400/429/413/403/echo-loop/long-poll-stuck/409/privacy/webhook/zero-by §10 6-subsection pre-deploy operator audit (tick-box checklist mirroring OPERATOR_CREDENTIALS.md style); §11 references (spec V3 §11.1/§32.2/§ HRDs + source paths). Cross-linked from existing docs/guides/messaging TELEGRAM.md (HRD-011 setup, preserved untouched). Wave 8 Track B — qaherald/internal/mtproto/ MTProto user-account wiring (gotd/td v0.144.0 vendored as submodules/gotd-td per §11.4.74 c Client interface (Connect / SendMessage / WaitForReply / WhoAmI / C gotd/td's telegram.Client with session-storage at ~/.config/herald mtproto.session (mode 0600, encrypted MTProto session blob). qaherald login/whoami/logout Cobra subcommands drive the AuthFlow: app_id + phone-from-env, 2FA password from env, OOB-code via stdin (TTY by default; HERALD_MTPROTO_ALLOW_NON_TTY=1 env override enables delegated-by piped flow for §11.4.98-compliant headless login). Conductor-driven end-to-end contribution = one-time paste of 5-digit Telegram OOB code at first-run login (fundamentally requires this; session persists thereafter for unlimited reruns). HF verified leak-free across all evidence (token-shape regex + 32-hex api_hash regex runtime artifact removed; .gitignore hardened).
HRD-139	task	high	Wave 8 Track B Test 1 — TestMTProto_Subscribe_AutonomousRound (qaherald/internal/lifecycle/mtproto_subscribe_test.go integration_mtproto). §11.4.98 replacement for the legacy TestSubscribe_LiveBotAPI Bot-API-only path which is fundamentally (a bot can never send-to-itself, so the legacy test required an operator typing in c drives the closed loop end-to-end via MTProto user-account: Client.SendMessage stimulus) → poll bot.getUpdates for delivery confirmation → assert message delivered within timeout. Honest-SKIP per §11.4.3 when MTProto creds absent OR session (never falls back to manual-dep path). Conductor-verified PASSING LIVE in 1. §11.4.98-pure interpretation (chain runs without human action during execution)
HRD-140	task	high	Wave 8 Track B Test 2 — TestMTProto_Wave6_AutonomousClosedLoop internal/lifecycle/mtproto_wave6_loop_test.go, build-tag integration_mtproto). §11.4.98 replacement for tests/test_wave6_live_loop.sh (which required manual operator typing). Sp listen as a child process with cmd.Dir = repoRoot (CRITICAL — pher docs/Issues.md relative to its CWD), pre-acks pending updates via consumePendingUpdates(t, botToken) helper (calls getUpdates(offset=max+1) to ACK stale chat messages BEFORE pher preventing TLS-connection-reset storms from prior runs). MTProto sends stimulus
HRD-141	task	high	

ID	Type	Criticality	Title
			based §11.4.98 autonomy assertion polls pherald's --qa-out-dir JSONL telegram.message (in) + cc.dispatch + cc.reply directions, matched on text string rather than message_id (MTProto's int64 message_id is a different namespace). API's chat-local message_id — §11.4.98 contract is "chain runs end-to-end with action", not "Claude generates perfect content"). PASSED LIVE in 48.58s.
HRD-142	task	high	Wave 8 Track B Test 3 — TestMTProto_Wave65_LifecycleAutonomy internal/lifecycle/mtproto_wave65_lifecycle_test.go, but integration_mtproto). §11.4.98 replacement for the legacy lifecycle --mode-interactive mode (which required operator typing per-scenario stimuli). Drives 3 Wave 6.5 fast-path scenarios — Help / Status / Continue — autonomously via M subtest spawns pherald + sends MTProto stimulus + asserts on the pherald journey telegram.message in + `cc.reply`
HRD-018	task	high	commons_constitution Go package: Evaluator interface + 12 event-class constitution_state/constitution_bindings migrations + bundle mode-ladder runtime config + constitution_audit write-through (the closing g Evaluator framework, emit helpers, bundle-hash, mode-ladder, state.Memory state.Postgres, ladder.Memory/ladder.Postgres, and migration 000006/000007/000008 were all coded in r1. v1.0.0 Batch A closed the remaining bluff: migration 000006 defined constitution_audit but NO code ever v Runner.Run set RunOutcome.Audited=true with nothing persisted. A AuditStore interface (audit.go) + state.MemoryAudit (append-only state.PostgresAudit (RLS-guarded INSERT-only into constitution append-only is enforced by the migration's no-update/no-delete RLS policies). F now writes a durable audit row for every CHANGED transition in ModeWarn/M recording old/new decision, old/new digest, bundle hash, mode-at-emission, and ModeEnforce — the EXACT emitted event ID (new IDEmitter optional integer PolicyViolationID/PolicyClearedID; emit returns the generated event NewRunner gained a required AuditStore arg (nil = hard error). §11.4.92 multi- RED (compile-fail before backends) → GREEN, -race -count=3 determining DB-level SELECT shows enforce rows carry emitted_event_id, warn rows NULL
HRD-026	task	middle	Constitution-bundle hash captureer — computes SHA-256 of rendered Constitution evaluation time; persists on every emitted event for replayability. The captureer Capture/CaptureBytes/Captureer) was coded in r1 and the hash already emitted-event metadata (bundle_hash hex). v1.0.0 Batch A completed the previous 32-byte bundle_hash is now stored on every constitution_state row constitution_audit row (verified by the DB-level SELECT ... encode(bundle_hash, 'hex') in the evidence dir showing a 64-hex SHA audit row), closing the §42.1.3 replay-correlation requirement.
HRD-027	task	middle	Mode-ladder runtime config (constitution_bindings table + admin REST flip allow/warn/enforce per binding per tenant without redeploy). The ModeLadder.Memory/ladder.Postgres (reading constitution_bindings time) were coded in r1. v1.0.0 Batch A added the admin REST surface: cherald modes — GET /v1/compliance/modes (list bindings + unbound-default compliance/modes/:rule_id (one binding's effective mode, enforce decision compliance/modes/:rule_id (flip allow/warn/enforce; records the operator mutated_by). JWT-gated via the existing commons_auth.GinMiddleware mounted in cherald serve (a single shared PG pool backs both compliance + bluff: 401 no-auth / 401 bad-tenant / 400 bad-mode (loud, never a silent default proof: a PUT flip to warn is reflected by a subsequent GET AND by ladder.C evaluation hot path — runtime config, no redeploy.
HRD-132	bug	high	

ID	Type	Criticality	Title
HRD-130	task	high	§11.4.85/§32.1 Runner idempotency-window FIX — concurrent same-key repla duplicate-dispatches a notification (FINDING from HRD-125 stress/chaos). Ro idempotency . go : 64 - 72 treated a Redis-SETNX-miss-with-no-PG-row as §32.1 "Redis-lies-PG-truths" design) while the events_processed archive written at Stage 7 (OutcomeRecorder . Process) — so concurrent same-k arriving in the Stage-2→Stage-7 window ALL dispatched (observed 2–4 of a 10 archival exactly-once via PG UNIQUE + ON CONFLICT DO NOTHING, but only bounded = duplicate-alert delivery under load). Fix: claim the idempotency BEFORE dispatch — INSERT events_processed ... ON CONFLICT moved to Stage 2 (the PG row, serialized by PRIMARY KEY(tenant_id, idemp now the AUTHORITATIVE exactly-once dispatch gate; 0-rows-affected = dupl dispatch), preserving the nil-Redis PG-only degrade. ALSO closed the latent ru CachedRcpt . WasReplay data race (mutated a possibly-shared *Receipt HRD-125 stress test was upgraded to assert sends==1 under the same 1000× — the stronger assertion HRD-125 deliberately could NOT make pre-fix (§11.4 on-discovery). §11.4.92 multi-pass: Pass-1 independent - race - count=3 ru storage green; Pass-5 sends==1 is code-true (RED proved before fix). wave3 retired (4f45f26) — Stage-7 Insert now redundant, M4's blast-radius moved i claim. §11.4.85 / §1.1 paired mutation gate for the GAP-3 stress+chaos suite — test test_stress_chaos_mutation_meta . sh (GAP-3 plan §5 / §6 T9). Pr hermetic stress/chaos detector is genuinely LOAD-BEARING via the canonical test_wave6.5_mutation_meta . sh idiom: . git / MUTATION_IN_PR dirty-tree + pre-existing-residue refusal pre-flights, run_paired (apply perl n marker landed → run detector EXPECT-FAIL → git checkout restore → b assert_restored vs HEAD → re-run EXPECT-PASS), and a final check scan asserting no MUTATED SC-M* markers leaked (§107.y). FIVE paired mu against a hermetic detector: SC-M1 drop ctx from claude_code dispatch . go exec . CommandContext → exec . Command → timeout-cancel test no long sleep → watchdog FAIL; SC-M2 swallow the claude_code parseReply deco truncated-reply test returns nil → FAIL; SC-M3 make inbound extractRepl default: panic instead of return ...err → degrade test panics → FAIL events mapErrorToStatus event_parser : 400→500 → input-corruption FAIL; SC-M5 swallow the commons_storage applyMigration disk-full wr ENOSPC reports success → silent-swallow guard FAIL. Runs ONE-AT-A-TIME FOREGROUND (§107.y + the 2026-05-27 concurrent-gate-residue memo). Co result: 6 PASS / 0 FAIL (5 mutations load-bearing).
HRD-129	task	high	§11.4.85 stress+chaos suite wired into the canonical anti-bluff harness script e2e_bluff_hunt . sh as new invariants E81-E88 (GAP-3 plan §4 / §6 T8) — ONLY append after E80 (E1-E80 logic untouched). Each invariant RUNS a RE stress/chaos test via the existing check helper AND cites the captured-evidence docs / qa / <run-id> / stress_chaos / as its §11.4.2/§11.4.5 positive-evid through a new sc_anchor helper that greps a SPECIFIC load-bearing value (artefact FAILs as a §11.4 PASS-bluff; absent evidence dir SKIPs-with-reason). runner exactly-once-archive + bounded dispatch (HRD-125); E82 / v1 / event + duplicate-key + auth-storm (HRD-123, anchor events / categorised _ er all_malformed_rejected_no_5xx); E83 / v1 / compliance fail-lou / v1 / safety_state concurrent-mutation (HRD-124); E85 pherald lis malformed-Raw panic-free + extract-degrade (HRD-126); E86 claude_code pro timeout-cancel + truncated-reply (HRD-127); E87 disk-full tagged-error ENOS E88 §12.6 host-mem headroom (HRD-128). All eight hermetic; live container-p fill / container-OOM variants are SKIP-with-reason inside the Go tests

ID	Type	Criticality	Title
			(HERALD_STRESS_LIVE_*=1 gated). Verified <code>bash -n clean</code> ; E81-E88 block deterministic; header bumped E1..E80→E1..E88. Full <code>scripts/e2e_bluff</code> reports 78 PASS / 0 FAIL (was 60; +18).
HRD-128	task	high	§11.4.85 container-orchestrated / resource-exhaustion stress+chaos tests (GAP-3 §6 T7 + §7 host-safety) — <code>commons_storage/resource_stress_chaos</code> (hermetic core) + <code>tests/test_resource_stress_chaos.sh</code> (LIVE core) HERMETIC PASS (×2 deterministic): (1) disk-full error propagation — a fake database. Database/Tx injects <code>syscall.ENOSPC</code> on the migration-UP REAL RunMigrations → <code>applyMigration</code> write path; asserts real write error NOT swallowed, error tagged <code>commons_storage: apply + exec u</code> errors. <code>Is(err, syscall.ENOSPC)</code> true, 0 migrations reported applied mem headroom — <code>HostMemHeadroom()</code> pre/post a 25× workload shows <code>used_fraction_delta</code> ≈ 0 (host needle did not move) = §12.6 compliance (deterministic SKIP without env): real bounded-scratch disk fill HARD-capped HERALD_STRESS_LIVE_DISK=1. LIVE container scenarios SKIP-with-real safety-gated): container OOM-confinement + connection-churn against a --me PG, gated HERALD_STRESS_LIVE_OOM=1 + a §12.6 pre-flight host-headroom REFUSES to add pressure when host used-fraction ≥ 60% (host was ~74% this SKIP). §12/§12.6 honoured: NO host-OOM, NO real-host-disk fill, NO host pro <code>test -race -count=3</code> green (PASS×3/PASS×3/SKIP×3); go vet clean §11.4.85 claude_code dispatch stress+chaos tests (GAP-3 plan §1 row 5 / §6 T6 <code>commons_messaging/dispatch/claude_code/dispatch_stress_chaos_test.go</code> (in-package, exercises unexported <code>bootstrapSession/parseReply</code>) + 5 committed hermetic shims <code>testd claude-*.sh</code>. Per §11.4.27 only the EXTERNAL claude CLI is faked via <code>New(binaryPath, ...)</code> seam; NO production source edited, NO new dep, NO invoked. STRESS: (1) N=24×25=600 concurrent <code>Dispatch</code> (--resume path every reply really parsed, every call resumed the SAME pre-seeded UUID, anchor p50=62.9ms/p95=96.4ms; (2) N=16 concurrent COLD-START bootstrap — all one anchor UUID persists, -race clean. CHAOS: (a) process-death <code>exit 13 -KILL \$\$</code> → tagged error, no hang (watchdog), partial stdout NOT accepted; 30s-sleep shim vs 800ms ctx → cancellation fires, Dispatch returns ~803ms not <<<HERALD-REPLY>>> → explicit decode <code>reply JSON</code> error, never silent FINDING-1: cold-start concurrent bootstrap is last-writer-wins by design (hone not <code>bootstrap_count==1</code> bluff). FINDING-2: timeout-cancellation latency claude being a single process (hardening direction = process-group kill, flagg owner). <code>go test -race -count=3</code> deterministically green ×3. §11.4.85 pherald listen inbound-path stress+chaos tests (GAP-3 plan §1 <code>pherald/internal/inbound/stress_chaos_test.go</code> (in-package unexported <code>extractReplyToMessageID</code>) + <code>pherald/cmd/pherald/listen_stress_chaos_test.go</code> (drives the REAL <code>runListen</code> loop). the channel <code>Subscriber</code> + <code>CC CodeDispatcher</code> boundaries are faked; the Dispatch routing switch + <code>runListen</code> fan-in + clean-shutdown run unmodified. STRESS N=16×50=800 concurrent <code>Dispatcher.Handle</code> — 0 errors, exactly-once receive (800/800), 0 goroutine leak, p50=0.41ms ~25k dispatch/s; (2) 400-event burst through <code>runListen</code> — nothing dropped, clean shutdown on cancel (<3s), no leak. CH Raw payloads (nil map, missing/nil/string/bool/slice/map <code>message_id</code>) → <code>extractReplyToMessageID</code> degrades to (0,err); valid int/int64/int32/float 64-dispatch corrupt-Raw flood panic-free (recover-guard), degraded <code>replyTo</code> subprocess fault (killed/truncated/marker-less/empty/unknown-action) → stage inbound: error, no reply leak, no hang; (c) once-only side-effect — 1000 red
HRD-127	task	high	
HRD-126	task	high	

ID	Type	Criticality	Title
HRD-125	task	high	same issue .open collapse to exactly-ONE durable sink row. <code>go test -r</code> deterministically green $\times 3$ on both packages. NO production source edited, NO §11.4.85 Runner 7-stage pipeline stress+chaos tests (GAP-3 plan §1 row 3 / §6 pherald/internal/runner/stress_chaos_test.go exercising the runner.Run (only the external boundary — PG events_processed store, chan SETNX seam — faked per §11.4.27; orchestrator + 7 stages run unmodified). S goroutines $\times 40$ iters concurrent same-key replay + fresh keys, -race clean. C duplicate flood @ 50 parallel under live atomic Redis SETNX; (a') same under fallback degrade; (b) hermetic PG-deadlock (SQLSTATE 40P01) injection on the events_processed Insert. SURFACED HRD-132: dispatch exactly-once was NO under concurrent replay (events_processed archived at Stage 7; observed 2–4 du of a $1000\times$ flood); tests asserted the honest code-true contract (archival exactly-dispatch) pre-fix, then upgraded to <code>sends==1</code> once HRD-132 landed the Stage <code>test -race -count=1</code> green. §11.4.85 Gin /v1/compliance (cherald) + /v1/safety_state (sherald) stress+chaos tests (GAP-3 plan §1 row 2 / §6 T3) — cherald/internal/compliance_stress_chaos_test.go + sherald/internal/safety_stress_chaos_test.go, each driving the REAL flavor handler process http test server dialed by net/http.Client through the production (cli.TOONMiddleware $\rightarrow$ auth $\rightarrow$ handler). Only the external boundary faked per §11.4.27. NO production source edited, NO new dep. STRESS (each flavor): $N=10\times 200=2000$ POSTs, -race clean — 0 errors, 0 5xx, all 200 (compliance $\sim 8.4k$ req/s, safety_state $\sim 13k$ req/s). CHAOS (each flavor): (a) concurrent state-mutation under load, -race-clean RWMutex instead; (b) auth storm — 1000 random tokens through the REAL commons_auth.GinMiddleware+HMAC(HS256) $\rightarrow 1000\times 401$, 0 bypass; <code>test -race -count=3</code> green $\times 3$.
HRD-124	task	high	Content-negotiation correctness under concurrency — every request alternates /application/toon vs application/json $\rightarrow$ 100% Content-Type match. Accept header (0 codec-bleed), every toon body real TOON wire bytes. CHAOS (each flavor): (a) PG-drop (errStore) $\rightarrow 2000/2000$ fail-loud 500 each naming the failed dependency "fabricated 2xx (a swallowed-error 200 {results: []} would falsely tell an observer "violations" — the §107 mode this gate forbids); safety_state has no hot-path DE code-true analogue (concurrent state-mutation under load, -race-clean RWMutex instead; (b) auth storm — 1000 random tokens through the REAL commons_auth.GinMiddleware+HMAC(HS256) $\rightarrow 1000\times 401$, 0 bypass; <code>test -race -count=3</code> green $\times 3$.
HRD-123	task	high	§11.4.85 Gin /v1/events stress+chaos tests (GAP-3 plan §1 row 1 / §6 T2) — internal/http/events_stress_chaos_test.go driving the REAL events handler (EventsHandler $\rightarrow$ runner.Run, all 7 stages) over a REAL http test server dialed by net/http.Client through the production Gin (cli.TOONMiddleware + auth gate). Only the external boundary (PG/Redis runner.NewFakeRunner) faked per §11.4.27. STRESS: $N=12\times 200=2400$ POSTs, -race clean — 0 errors, 0 5xx, all 202; p50=0.706ms $\sim 13,322$ req/s. CHAOS (each flavor): (a) duplicate-key-under-load (1200 POSTs, per-worker keys) $\rightarrow$ graceful degrade, (b) input-corruption (8 MiB random, truncated JSON, non-UTF8, JSON array, missing field, empty) $\times 5 \rightarrow$ every body a tagged event_parser: 4xx, 0 panic, 0 5xx; (d) 1000 random tokens through the REAL commons_auth.GinMiddleware $\rightarrow 1000\times 401$, 0 bypass; (a) live PG-drop $\rightarrow$ SKIP-with-reason (proven hermetic FINDING ($\rightarrow$ HRD-132 at HTTP layer): a SHARED-key cross-worker concurrent triggers the REAL runner.go:132 CachedRcpt.WasReplay data race (HRD-132). E82 anchor re-captured as all_malformed_rejected_no_5xx transport-rejection-of-8-MiB-body is valid).
HRD-122	task	middle	§11.4.85 stress+chaos shared scaffold — Go-native commons/stresschaos package (GAP-3 plan §2 / §6 T1). Provides: a bounded concurrent load-runner (workers $\times M$ iterations of a closure via stdlib sync.WaitGroup, no new dependencies) rank percentile math (Percentiles $\rightarrow$ {p50, p95, p99, max, min, count}).

ID

Type Criticality Title

HRD-114 task high

latency.json + latency_histogram.csv; an evidence-dir writer creates <run-id>/stress_chaos/<surface>/; and a best-effort host-memory (HostMemHeadroom — vm_stat+sysctl hw.memsize on darwin, /proc/meminfo on linux) for the §12.6 60%-ceiling proof (degrades to probe-unavailable failure). Self-tested: 8 unit tests (percentile correctness on a known 1..100ms data creation, bounded-goroutine proof, host-mem best-effort, §12.6 ceiling predicate test -race -count=1. Lowest sensible layer per CLAUDE.md layering (not importable by every flavor).

Wave 7 T5 — multi-channel pherald listen (HERALD_CHANNELS fans → goroutines → 1 dispatcher). pherald/internal/inbound/dispatcher Wave-6 TgramReplier interface (Telegram-native int64 chatID/int64 reply signature) → generic inbound.Replier with SendReply(ctx, recipient commons.Recipient, body string, replyToID string, attachments []commons.Attachment) (string, error) — recipient + string ids satisfy it. Config.TgramReply → Config.Reply; NewDispatcher new Dispatcher.reply field + the Handle "reply" branch + both fastPath migrated to build a commons.Recipient{Channel, ChannelUserID} strconv.Itoa(replyToID) (" " when 0). pherald/cmd/pherald/listenConfig.Subscriber(singular) → Subscribers map[string]InboundHandler error; listenConfig.Replier inbound.Tgram inbound.Replier; new loadEnabledChannels() parses HERALD_CHANNELS (comma-split, trim, dedupe; unset/empty → ["tgram"], Wave-6 single-channel preserved); new perChannelConfig(name) reads namespaced env (HERALD_TGRAM_BOT_TOKEN/HERALD_TGRAM_CHAT_ID, HERALD_SLACK_BOT_TOKEN/_APP_TOKEN/_CHANNEL_ID) → channel load on missing required creds); loadListenConfigFromEnv resolves each via the channels.New(name, cfg) registry (T2), builds Subscribers[name]=ch.Subscribe + a channelReplier{ch} shim channels.Channel.SendReplyGeneric → inbound.Replier.SendReply two intentionally-differently-named generic methods bridged) per channel, and a channelRouter that dispatches each reply to the adapter for recipient.Channel load no replier for channel %q on unknown — never silently dropped fans in one goroutine per subscriber (manual sync.WaitGroup + buffered errgroup new dep — errgroup-equivalent semantics) all feeding ONE dispatcher; the first subscriber error cancels siblings + surfaces (T11 chaos fail-loud). tgram is blank listen.go for its init() registry registration. journal.go journalingReplier to wrap inbound.Replier (records channel+chat_id+reply_to_message strings; JSONL kind kept tgram.send_reply for transcript back-compat). 1 resolution (Wave 7 plan Step 4): inbound.Replier.SendReply keeps the name (the inbound package's own, independent of channels.Channel.SendReplyGeneric); production wiring bridges via channelReplier shim so the journal decorator + dispatcher both stay channels evidence: new TestRunListenMultiChannelFanIn (2 stub subscribers publish 1 event) asserts BOTH messages dispatched AND both replies recorded (fake)"] >= 2) — NOT "both goroutines started"; mutation-proven load-balance the fan-in loop → seen == map[] → FAIL). Existing single-channel TestListenWiresHandlerAndExitsOnCancel + TestRunListenRejectsBadConfig migrated to the one-entry Subscriber PASS. recordingReplier/stubReplierJournal stubs (dispatcher_test journal_test) migrated to the generic signature; numeric chat_id/reply assertions decode.go build ./pherald/... + go test -race -count=1 .

ID

Type Criticality Title

HRD-113 task high

pherald/... ./commons_messaging/... all green; go vet clean; clean.

Wave 7 T4 — generalize the bot self-filter via BotSelfIdentity (channel-agnostic) commons_messaging/channels/selffilter.go adds the Raw-map (RawSenderIsBot/RawSenderIdentityKind/RawSenderIdentity) StampSender(raw map[string]any, isBot bool, kind IdentityKind, value string) (records the sender's native identity into ev.Raw; nil-map if not) IsSelfEcho(ev commons.InboundEvent, self SelfIdentity IdentityKind) §32.9 anti-echo-loop guarantee made channel-agnostic — compares the right native IdentityKind: username for Telegram, user_id for Slack, address for Slack; deliberately narrow: a DIFFERENT bot is KEPT (multi-bot collaboration is real) — the §107 anchor that the filter is not over-broad); a human is KEPT; an empty self never echoes (conservative KEEP → surfaces as duplicate traffic, not silent loss) Subscribe refusing to boot on empty self). commons_messaging/channels/subscribe.go migrated: captures self := channels.SelfIdentity{channels.IdentityUsername, Value: bot.Me.Username} once (keeping the existing empty-username boot-refusal echo-loop guard) and routes handlers (OnText/OnPhoto/OnDocument/OnVoice) through a new stampAndIsSelfEcho(ev, msg, self) helper that stamps the Telegram StampSender.IsBot+@username into ev.Raw and delegates to channels.IsSelfEcho. The four Wave-6 shouldDropBotSelf(msg, selfUsername) early-return native shouldDropBotSelf func + its TestSubscribeBotSelfFilter SelfFilterForTest (export_test.go) are PRESERVED BYTE-IDENTICAL. The paired-mutation gate (tests/test_wave6_mutation_meta.sh) anchored by perl -i -0pe regex on shouldDropBotSelf's body, so keeping it byte-identical. M1 load-bearing (verified: the M1 regex still matches + applies its W6-M1 migration). The live path and the gate now express the SAME §32.9 invariant — the generalized IsSelfEcho, the gate on the tgram-native shape. §107 evidence: TestIsSelfEchoUsername proves all three username cases — bot-own DROPPED, different-bot KEPT, human KEPT; TestIsSelfEchoUserID proves the Slack cases (matching bot id DROPPED, different id KEPT); TestIsSelfEchoEmptySelfNeverEchoes proves empty-self KEEP; the test builds events via the production StampSender (DRY — the test can never drop key contract); TestSubscribeBotSelfFilter still PASSES (Wave-6 sha256 test -race -count=1 ./commons_messaging/... ./pherald/... go vet clean. Live runtime evidence (real Telegram getMe self-identity + own self) the closed loop) lands via Wave 7 T9 e2e invariant.

HRD-112 task middle

Wave 7 T3 — per-channel inbox subdirs ~/.herald/inbox/<channel>/<sha256>.<ext>. New file commons_messaging/channels/inbox/channel-agnostic InboxDir(channel string) (string, error) (returns ~/.herald/inbox/<channel>/, MkdirAll 0700, empty-channel error) + WriteContentAddressed(channel, mime string, r io.Reader) (path, sha256Hex string, err error) (streams r into <dir>/<sha256>.<ext> via io.MultiWriter(tmp, sha256.New()) — no full payload — then atomic temp+rename; idempotent: if the content-addressed file the .part temp is dropped and the existing path returned unchanged; closes r) MimeToExt(mime string) (the Wave-6 tgram-private mimeToExt switch moved VERBATIM to package level so every adapter shares one map). commons_messaging/channels/tgram/attachments.go DownloadAttachment now does Telegram-specific bot.File(&telebot.File{FileID}) fetch then delete hash-rename to channels.WriteContentAddressed(string(common.ChannelName, sha256Hex, ext))

ID

Type Criticality Title

mime, rc) — so tgram attachments land under ~/ .herald/inbox/tgram channel subdir, was the flat ~/ .herald/inbox/ in Wave 6). The tgram-priv + the crypto/sha256/encoding/hex/io/os/path/filepath imports from attachments.go (the helper owns them now); the method-form (*Adapter).DownloadAttachment(T1) delegates to the package func forms route through the shared helper. §107 evidence: TestInboxDirIsPer InboxDir("tgram") ends in .../inbox/tgram AND InboxDir("t InboxDir("slack") (no flat-inbox regression); TestWriteContentAddressedHashesAndIsIdempotent pins all the classes on real on-disk state — (a) path shape under inbox/slack/ named < re-read on-disk bytes byte-equal the payload (sha in path == sha of content), (c) content returns same (path, sum) and leaves zero .part residue (real os .R the existing Wave-6 TestDownloadAttachmentContentAddressed (u tgram/ path segment) still PASSES end-to-end against a httptest Bot API routing change is behaviour-preserving. go test -race -count=1 ./ commons_messaging/... ./pherald/... ./qaherald/... all g assumption regressions); go vet clean. Live runtime evidence (real Telegram landing under inbox/tgram/) lands via Wave 7 T9 e2e invariant E83.

Wave 7 T2 — channel registry (name→constructor) + init()-based registrati commons_messaging/channels/registry.go adds Register(name Constructor)/New(name, Config) (Channel, error)/Names the ErrUnknownChannel sentinel, the channel-agnostic Config struct (Ch AppToken/Target/BaseURL/Extra), and the Constructor func(Co (Channel, error) type — a sync.RWMutex-guarded map[string]C Register panics on duplicate (a typo in two adapters' init() would silently §107 bluff class this framework forbids; mirrors the qaherald scenario.Reg New wraps ErrUnknownChannel(fmt.Errorf("%w: ...")) for unk callers (pherald listen) MUST surface it; a silent no-op channel is a §107 commons_messaging/channels/tgram/tgram.go gains an init() channels.Register(string(common.ChannelTelegram), .. constructor builds NewAdapterWithBaseURL(cfg.Token, cfg.Targ cfg.BaseURL) when cfg.BaseURL != "" (httptest seam) else NewWithCreds(cfg.Token, cfg.Target) — so a blank import self-r adapter. §107 evidence: TestRegistryResolvesRegisteredChannel actually constructs + returns a working Channel(c.Name()=="fake-rt" TestRegistryUnknownChannelErrors proves errors.Is(err, ErrUnknownChannel) for unknown names; TestRegistryDuplicate the duplicate-Register panic; TestTgramRegisteredViaInit(blank tgram") proves the init() actually registered tgram in Names() (NOT a failing before the init() landed: Names()=[dup-rt fake-rt] without after). The shared fakeChannel test stub uses SendReplyGeneric (the T interface method name, NOT SendReply). go test -race ./ commons_messaging/... green (tgram regression-free); all 16 workspace go vet ./commons_messaging/... clean. Live runtime evidence (real by name + round-trip) lands via Wave 7 T9 e2e invariants E81/E82.

Wave 7 T1 — extract commons_messaging/channels.Channel interf New package commons_messaging/channels/ defines Channel (emb commons.Channel — Name/Capabilities/Send/Subscribe/HealthCheck — p inbound-runtime methods Telegram implicitly defined: SendReplyGeneric BotSelfIdentity returning SelfIdentity{Kind, Value}, Downloa the SelfIdentity value type, and the IdentityKind constants (Identi IdentityUserID/IdentityAddress). *tgram.Adapter gains BotS

HRD-111 task high

HRD-110 task high

ID

Type Criticality Title

HRD-101 task high

(getMe via ensureBot; empty-username → §107 echo-loop-hazard error), SendReply (string→int64/int adapter over the native Wave-6 SendReply), and a method DownloadAttachment (routes the package-level func through ensureBot's l _ channels.Channel = (*tgram.Adapter)(nil) compiles. Naming from plan Step 4 text resolved + documented in channel.go package doc: the method is named SendReplyGeneric (not SendReply) so tgram satisfies touching its native, §107-mutation-gate-anchored int64/int SendReply (whose generic signature is T5's scope) — keeping T1 a pure refactor with zero cross-p §107 evidence: var _ channels.Channel = (*tgram.Adapter)(nil) (r assertion in channel_test.go (a renamed method / drifted signature breaks TestTgramSatisfiesChannel + TestSelfIdentityType PASS; full -race ./commons_messaging/... green (tgram regression-free — pure all 16 workspace modules build; go vet ./commons_messaging/... c evidence (real getMe self-identity + reply round-trip) lands via Wave 7 T9 e2e i Wave 6.5 ticket-lifecycle layer — §32.6 deterministic classifier (Bug:/Issue: Implementation:/Investigation:/Query:/Question:/Help:/Status/Continue:/Done:/Resolve:/Reopen:/Override: prefix routing with Type+Criticality+Confidence scores; natural-language no-prefix → c DocsIssueOpener (§32.9 — real docs/Issues.md HRD-NNN allocator, con lockfile) + RunnerEventEmitter (§33.7 — direct runner.Run → CloudEvent HTTP loopback) + Help:/Status:/Continue: fast-path handlers (bypass second latency — S2 proved 0.74s, no CC bootstrap overhead) + Done:/Reopen Issues) Fixed migration (rollback-via-sha256 test) + tgram.SendReply ou fan-out (multipart dispatch) + wiring all of these into pherald listen. §10 (docs/qa/HRD-101-lifecycle-2026-05-23T03-16-17-w6.51 liv language query — real Opus CC dispatch ~31s incl. bootstrap, threaded reply (v inbound message_id=30 → reply sent_message_id=31 with reply_to_message_id=30); S2 Help: fast-path — classifier Type:he Confidence: 1, CC dispatch ABSENT (event absence + 0.74s latency prove BuiltinHelp catalogue text returned (reply_to_message_id=32 → sent_message_id=33); transcript.jsonl captures both directions of all four query, Help:, Status: also exercised live). Reproducible test harness tests/ test_wave6.5_lifecycle.sh covers 15 scenarios (Bug:/Task:/Query: + Done:/Reopen: + classifier edge cases). Spec V3 r13: §32.6/§32.9/§33.6/§33.7 s Tag v0.5.0.

HRD-100 task high

Wave 6 — pherald inbound runtime + Claude Code headless bridge (code-doc c Cobra subcommand pherald listen runs the closed-loop runtime: Telegra long-poll (OnText/OnPhoto/OnDocument/OnVoice) + bot self-filter (bot anti-echo guard captured at Subscribe construction; refuses to start if getMe re username) + sha256 content-addressed attachment download into ~/ .herald <sha256> . <ext> (idempotent — duplicate downloads do not re-write) + Cl dispatch with Opus pinned in the spawned argv (claude --model claude resume <UUID> --print <envelope>) + envelope pre-text — verbatim wording ("We have received new message from our communic channel <channel-name>. <classification sentence>. <a list>") preceding the <<<HERALD-DISPATCH-v1>>> structured block + REPLY>>> parser with action routing (reply default / issue.open / ev tgram.Adapter.SendReply(ctx, chatID, text, replyToID, attachments) wiring reply_to_message_id from the original messa out-dir <path> flag journals every inbound + outbound event as JSONL p 12 atomic Wave-6 commits T1=ad87d7f (HERALD_PROJECT_NAME reso + .env.example), T2=0d55274 (Opus model pin), T3=14626f7 (envelope pro

ID	Type	Criticality	Title
			T4=b4018a3 (bot self-filter), T5=d9156f1 (OnPhoto/OnDocument/OnVoice download), T6=a22a054 (pherald listen Cobra subcommand), T7=a3b (pherald/internal/inbound/ Dispatcher + action routing), T8=5f92f (tgram.SendReply with reply_to_message_id), T9=f7913b8 (live script tests/test_wave6_live_loop.sh), T10a=520e0c6 (--qa-on JSONL journaling middleware), T11=01bfd1a (e2e_bluff_hunt E63-E70 inva T12=96c7c6b (Wave 6 mutation gate tests/test_wave6_mutation_m paired hermetic mutations). §107 evidence: 8 new e2e invariants E63-E70 land currently SKIP with documented reasons — E63 lifecycle / E64 bot self-filter / sha256 / E66 envelope pre-text / E67 Opus argv / E68 reply_to_message_ E69 action routing issue.open / E70 live closed-loop). Conversion SKIP → PAS when the operator exports HERALD_TGRAM_BOT_TOKEN + HERALD_TGRAM HERALD_CLAUDE_BIN and runs bash tests/test_wave6_live_loc out-dir docs/qa/HRD-100-<run-id>/. Tag v0.4.0 is DEFERRED T10b live evidence per §107.x) — code-doc closure ships here. pherald /v1/events Runner wiring (REST API surface via Gin Gonic per sp stage event-ingest pipeline lands as 7 concrete structs under pherald/ inter (EventParser, IdempotencyChecker, TenantResolver, PolicyGate, SubscriberRes ChannelDispatcher, OutcomeRecorder) orchestrated by runner .Runner .Ru internal/http/events .go exposes the live HTTP handler; pherald/ {main, stubs} .go lazily build the Runner + JWT verifier inside the serve s other paths still work without HERALD_PG_DSN/HERALD_AUTH_MODE. HTTP Accepted + Receipt JSON on fresh event, 200 + X-Herald-Replay: true missing/invalid JWT, 400 on parse failure. §107 evidence: 23/23 runner package stage + 3 integration scenarios: happy/duplicate/deny); 6 new e2e invariants E3 gate-skipped when absent — honest SKIP-with-reason per §11.4.3); mutation g test_wave3_mutation_meta .sh gains M2 (Duplicate=false uncondition M4 (skip events_processed.Insert → E38/E42 break). M3 SKIP-with-reason per deny-evaluator e2e. Commits T1..T10: eb9b2f4, 40ad60f, 2fdb7b5, 4c4 5468897, 6ea29a4, be71db1, c7b89a4, (this commit). Telegram channel adapter live integration — telebot.v3 vendored (submodule HealthCheck + Send + Subscribe + outbound_delivery_evidence persistence all §107 bluff guards. Closed atomically with operator-session live evidence 2026- delivery message_id=5 from @atmosphere_worker_bot to chat 2057 (validated via getChat) using the extended wizard CLI's --bot-token (en HERALD_TGRAM_BOT_TOKEN) + --chat-id (validated via getChat l --non-interactive flags. Token→getMe→getChat→sendMessage chain end in commit 140a2f1 session. commons_auth/ JWT verifier — HMAC + JWKS hybrid + Gin middleware + c vendored Herald-internal per §11.4.74 catalogue-check no-match. Catalogue ev catalogue-checks/HRD-099-commons-auth.md. Renumbered from 3a executing-time anchor) in the post-Wave-3a doc-cleanup pass; Wave 3a com dbbea95 . . 0cc6fad keep the old HRD-093 in their commit messages as his sherald /v1/safety_state live — process-local Aggregator + background tick) + Handler with JWT gate via commons_auth.GinMiddleware. Replaces the §107 evidence: e2e E46 (no-auth → 401), E47 (valid HMAC → 200 + current_ last_destructive_op=null + uptime_seconds>=1 + open_events=0). Paired muta mem Aggregator) proves E47 catches the regression. E48 SKIP-with-reason — trigger awaits HRD-033 body. cherald /v1/compliance live — paginated + filter-aware constitution surface with JWT gate via commons_auth.GinMiddleware. Backed by ConstitutionStore.ListQuery extended with Since/Until/Offset (commit 627643
HRD-016	task	middle	
HRD-011	task	middle	
HRD-099	task	low	
HRD-098	task	middle	
HRD-028	task	low	

ID	Type	Criticality	Title
			e2e E43 (no-auth → 401), E44 (valid HMAC + empty tenant → 200 + total=0 + page_size=50). E45 SKIP-with-reason — cross-binary cherald-reads-pherald-w Wave 3b (Runner not live yet).
HRD-092	task	middle	commons/cli/ shared CLI scaffold (Wave 2) — NewRootCmd + VersionCmd + Middleware hook) + StubCmd + healthz/readyz/metrics handlers. Vendored as §11.4.74 catalogue-check no-match. As-built evidence: §44.M of spec V3 r8 + checks/HRD-092-commons-cli.md.
HRD-093	task	middle	sherald flavor scaffold — System Herald binary serving :24793 with cli.ServeC (HRD-033/034/040/046/056) + /v1/safety_state 501 stub → HRD-098.
HRD-094	task	middle	cherald flavor scaffold — Constitution Herald binary serving :24792 with cli.Se stubs + /v1/compliance 501 stub → HRD-028.
HRD-095	task	low	bherald flavor scaffold — Build Herald CLI-only binary with 3 §43 stubs (HRD
HRD-096	task	low	rherald flavor scaffold — Release Herald CLI-only binary with 3 §43 stubs (HR
HRD-097	task	low	iherald + scherald flavor scaffolds — Incident Herald serving :24794 with /v1/w HRD-024 + Scheduled-audit Herald CLI-only with 1 §43 stub (HRD-047). Pair has zero §43 stubs.
HRD-012	task	middle	Claude Code dispatcher live integration — <code>claude --resume <UUID> - <envelope> exec + <<<HERALD-REPLY>>> JSON parse + claude_cod persistence per §33. §107 evidence: Plan 2 Task 6 commit 702b5a3 (live PASS claude CLI round-trip, structured reply parsed, Outcome+Summary non-empty) 4718c0e (live PASS 36.23s — session_state upsert under HeraldSystemTenan equality assertions on session_uuid + anchor_path + last_response JSONB round HRD-085..HRD-090 stay open for upstream-defined TaskRepository methods m Dispatch+session hot path.</code>
HRD-010	task	middle	commons_storage live wiring — pgx pool + RLS-enforcing WithTenantContext fixed RLS-bypass bug via E14 falsifiability) + 9 migrations + background queue (digital.vasic.background bound via pgxTaskRepository) + Redis ACL (digital.v pherald migrate up/status/down/validate subcommand + 3 new §107 e2e invaria + HRD-085..090 registered for queue-repository stubs
HRD-080	task	low	Refine I6 inheritance-gate invariant from blanket ".gitmodules-forbidden to constitution/entry in .gitmodules" — paired meta-test test_i6_refinement_meta.sh with 3 anti-bluff subtests. Enables Found Helix-stack submodule installs.
HRD-017	task	middle	Propagate Universal §11.4.73 (main-spec versioning + revision discipline) and (submodule-catalogue-first discovery) into parent constitution Constitution.md AGENTS.md
HRD-014	task	middle	pherald CLI scaffold — Cobra root + version + 5 stubbed subcommands; pher --json returns canonical build info
HRD-013	task	middle	commons_messaging + null:// adapter — full §11.0 Channel contract impl with fail_rate/latency/ceiling URL params + 8-case unit test suite
HRD-009b	task	low	commons_prefix module — §8.2 deterministic 3-letter prefix algorithm with Ca collision-resolution via fnv1a32 + table-driven tests
HRD-009	task	middle	commons module — full §11.0 Go type contract (Channel + Capabilities + Deli OutboundMessage + Body + Attachment + Recipient + Action + Priority + Rec InboundHandler + InboundEvent + Subscriber + SubscriberAlias + CloudEvent TraceContext + Branding + ChannelID + PreferenceSet + ...) + Clock abstraction helper + DefaultBranding factory
HRD-007	task	middle	V3 r3 cross-doc sync + tracking-doc scaffold (Issues/Fixed/Status/Status_Summ Issues_Summary/Fixed_Summary/CONTINUATION)

ID	Type	Criticality	Title
HRD-006	task	middle	V3 r2 flavor refinement — 9 flavors × per-channel interaction tables
HRD-005	task	middle	V3 r1 operator-product layer (§31..§36 + §18.2 expansion)
HRD-004	task	middle	V2 r3 — Go type contract closure + operational ops detail
HRD-003	task	middle	V2 r2 — close prose ^[?] definition gaps + add operational guidance
HRD-002	task	middle	V2 r1 — architectural authoring (CloudEvents/Watermill/OTel/RLS/SLSA/9 fla
HRD-001	task	middle	V1 — initial MVP specification + Review section + Recommendations